

네트워크 3 - UDP

남궁명수

[UDP]

UDP(User Datagram Protocol)는 TCP와 함께 전송 계층에서 사용되는 프로토콜이다.

TCP가 연결을 설정하고 데이터 전달을 확인하는 방식이라면, UDP는 연결 설정과 전달 확인 없이 데이터를 바로 전송하는 비연결형 프로토콜이다.

TCP: 연결 설정 → 데이터 전송 → 전달 확인 → 연결 종료

UDP: 데이터 전송

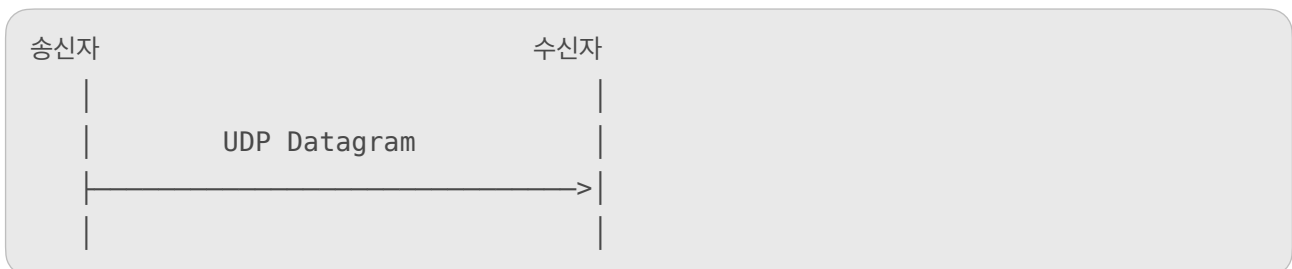
UDP의 핵심 목적은 신뢰성 관련 기능을 최소화하여 단순하고 빠르게 데이터를 전달하는 것이다.

[1] [UDP의 특징]

[비연결형 프로토콜]

UDP는 TCP처럼 통신 전에 3-Way Handshake를 수행하지 않는다.

송신자는 상대방이 실제로 존재하는지, 데이터를 받을 준비가 되었는지 확인하지 않고 바로 데이터를 전송할 수 있다.



TCP는 통신 전에 연결을 설정한다.

SYN → SYN+ACK → ACK → 데이터 전송

UDP는 연결 설정 없이 곧바로 전송한다.

데이터 전송

여기서 비연결형이라는 것은 IP주소와 포트를 사용하지 않는다는 뜻이 아니다.

UDP도 데이터를 전달하려면 목적지 IP 주소와 포트 번호가 필요하다.

목적지 IP → 어느 컴퓨터인가?

목적지 Port → 그 컴퓨터의 어느 프로세스인가?

다만 TCP처럼 클라이언트와 서버 사이의 연결 상태를 지속적으로 관리하지 않는다는 의미다.

[2] [데이터 도착을 보장하지 않는다]

UDP는 데이터를 전송한 뒤 수신자가 정상적으로 받았는지 확인하는 ACK 기능을 기본적으로 제공하지 않는다.

송신자 → 데이터 전송

송신자 → 전달 여부를 확인하지 않음

따라서 네트워크에서 데이터가 손실되어도 UDP 자체는 이를 복구하지 않는다.

Datagram 1 → 도착

Datagram 2 → 손실

Datagram 3 → 도착

수신자는 2번 데이터가 손실됐더라도 UDP만으로는 재전송을 요청하지 않는다.

필요하다면 애플리케이션이 직접 다음 기능을 구현해야 한다.

- 응답 메시지
- 시퀀스 번호
- 재전송
- 타임아웃
- 중복 제거

즉, UDP에서 신뢰성이 필요하다면 애플리케이션 계층에서 구현할 수 있다.

[3] [데이터 순서를 보장하지 않는다]

UDP는 먼저 보낸 데이터가 먼저 도착한다고 보장하지 않는다.

전송 순서: 1 → 2 → 3

도착 순서: 2 → 1 → 3

각 UDP 데이터그램은 네트워크에서 서로 다른 경로를 거칠 수 있다.

따라서 전송 순서와 도착 순서가 달라질 수 있다.

UDP 자체에는 TCP처럼 시퀀스 번호를 이용해 순서를 재조립하는 기능이 없다.

순서가 반드시 필요하다면 애플리케이션 데이터에 별도의 번호를 포함해야 한다.

```
Packet {  
    sequence: 1,  
    data: "..."  
}
```

[4] [재전송하지 않는다]

TCP는 데이터가 손실되었다고 판단하면 해당 데이터를 재전송한다.

UDP는 데이터 손실을 확인하는 ACK와 재전송 타이머를 기본적으로 제공하지 않으므로

손실된 데이터를 자동으로 재전송하지 않는다.

TCP:

데이터 전송 → ACK 없음 → 손실 판단 → 재전송

UDP:

데이터 전송 → 종료

이 때문에 UDP는 신뢰성은 낮지만, 오래된 데이터를 다시 보내느라 실시간성이 떨어지는 것을 피할 수 있다.

예를 들어, 실시간 음성 통화에서는 조금 전에 손실된 음성을 뒤늦게 재전송받는 것보다 현재 음성을 계속 재생하는 편이 더 중요할 수 있다.

[5] [흐름 제어와 혼잡 제어를 기본 제공하지 않는다]

UDP와 TCP의 다음 기능을 기본적으로 제공하지 않는다.

- 수신자의 처리 능력을 고려하는 흐름제어
- 네트워크 상태를 고려하는 혼잡 제어

따라서 UDP 송신자가 데이터를 너무 빠르게 전송하면 다음 문제가 발생할 수 있다.

송신 속도 > 수신 처리 속도

↓

수신 버퍼 포화

↓

데이터그램 손실

또는

송신량 > 네트워크 처리량

↓

라우터 큐 포화

↓

네트워크 혼잡 및 패킷 손실

따라서 UDP 기반 프로그램을 개발할 때는 필요에 따라 전송 속도를 애플리케이션에서 조절한다.

UDP 위에서 동작하는 QUIC 같은 프로토콜은 애플리케이션 영역에서 신뢰성, 재전송, 흐름 제어, 혼잡 제어 등을 구현한다.

UDP가 해당 기능을 제공하지 않는다는 것이지, UDP를 사용하는 프로그램이 해당 기능을 구현할 수 없다는 뜻은 아니다.

QUIC(Quick UDP Internet Connections): Google이 개발하고 현재 IETF 표준인 UDP 기반 전송 프로토콜

[6] [메시지 경계를 보존한다]

UDP는 데이터그램 단위로 데이터를 전달한다.

송신자가 UDP 데이터를 두 번 전송하면 수신자가 각각의 데이터그램 단위로 받는다.

```
송신자:
send("Hello")
send("World")

수신자:
recv() → "Hello"
recv() → "World"
```

각 send()로 전송한 데이터가 하나의 데이터그램으로 다뤄진다.

반면, TCP는 바이트 스트림이므로 메시지 경계를 보존하지 않는다.

```
TCP 송신:
send("Hello")
send("World")

TCP 수신 가능 결과:
recv() → "HelloWorld"
```

또는

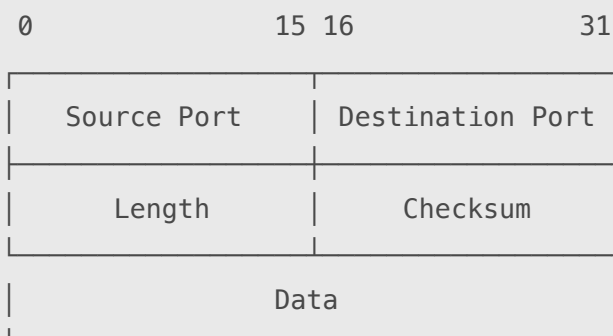
```
recv() → "Hel"
recv() → "loWorld"
```

TCP에서는 애플리케이션이 메시지 경계를 직접 구분해야 하지만 UDP는 데이터그램 경계를 보존한다.

다만 수신 버퍼가 데이터그램보다 작으면 남은 데이터가 다음 recvfrom()에서 이어지는 것이 아니라 잘릴 수 있다. 따라서 수신 버퍼 크기를 적절하게 지정해야 한다.

[7] [헤더가 단순하다]

UDP 헤더는 기본적으로 8바이트이며 다음 네 가지 필드로 구성된다.



필드	크기	의미
Source Port	16비트	송신 프로세스의 포트
Destination Port	16비트	수신 프로세스의 포트
Length	16비트	UDP 헤더와 데이터를 합친 전체 길이
Checksum	16비트	전송 중 오류 검출
전체 UDP 헤더	8바이트	고정 크기

TCP 헤더는 기본 20바이트이며 옵션이 붙으면 더 커질 수 있다.
반면 UDP는 8바이트로 고정되어 있어서 구조가 단순하다.

단, UDP 헤더가 작다고 해서 모든 상황에서 TCP보다 반드시 빠르다는 뜻은 아니다.
실제 성능은 네트워크 상태와 애플리케이션 구현 방식에도 영향을 받는다.

[8] [오류를 검출하지만 복구하지 않는다]

UDP에는 체크섬이 있어 데이터가 전송 중 손상됐는지 검사할 수 있다.

오류 검출 가능
오류 복구 불가능

체크섬 검사 결과 데이터가 손상된 것으로 판단되면 데이터그램을 폐기할 수 있다.

하지만 UDP 자체는 폐기된 데이터를 다시 보내달라고 요청하지 않는다.

UDP Checksum:
“데이터가 손상됐는가?” → 검사 가능
“손상됐으니 다시 보내라.” → 처리하지 않음

참고로 UDP 체크섬은 일반적인 IPv4 UDP에서는 0으로 설정해 사용하지 않을 수 있지만,
IPv6 UDP에서는 원칙적으로 사용해야 한다.

[9] [브로드캐스트와 멀티 캐스트가 가능하다]

UDP는 하나의 송신자가 여러 수신자에게 데이터를 보내는 통신 방식에 적합하다.

유니캐스트: 한 송신자가 한 수신자에게 전송

송신자 → 수신자 1명

브로드캐스트: 같은 네트워크에 있는 모든 장치에 전송

송신자 → 같은 네트워크의 모든 장치

멀티캐스트: 특정 멀티캐스트 그룹에 참여한 장치들에게 전송

송신자 → 특정 그룹의 여러 장치

TCP는 연결마다 일대일 연결 상태를 관리하므로 브로드캐스트나 멀티캐스트 방식으로 사용하지 않는다.

[10] [UDP가 사용되는 곳]

UDP는 일부 데이터가 손실되더라도 속도와 실시간성이 중요한 통신에 적합하다.

[DNS]

DNS 질의와 응답은 크기가 작고 빠르게 처리해야 하므로 전통적으로 UDP를 많이 사용한다.

클라이언트 → 도메인의 IP 주소 질의

DNS 서버 → IP 주소 응답

다만 DNS가 항상 UDP만 사용하는 것은 아니다. 응답 크기, 영역 전송, 통신 방식 등에 따라 TCP도 사용한다.

[실시간 음성 및 영상]

실시간 통신은 손실된 과거 데이터보다 현재 데이터를 제시간에 받는 것이 더 중요할 수 있다.

- 인터넷 전화
- 화상 회의
- 실시간 스트리밍
- WebRTC

예를 들어, 1초 전 음성이 손실됐다고 해서 지금 다시 재생하면 통화가 끊기고 지연될 수 있다. 이런 환경에서는 일부 손실을 허용하고 현재 데이터를 계속 처리하는 편이 적절하다.

[온라인 게임]

실시간 위치, 방향, 행동 정보는 빠른 전달이 중요하다.

1초 전 캐릭터 위치를 재전송 → 이미 오래된 정보일 수 있음

따라서 일부 게임은 실시간 상태 전송에 UDP를 사용한다.

다만 아이템 구매나 로그인처럼 반드시 정확히 처리해야 하는 데이터에는 별도의 신뢰성 처리나 TCP를 사용할 수 있다.

[DHCP]

장치가 아직 자신의 IP 주소나 DHCP 서버의 주소를 정확히 알지 못하는 초기 단계에서 브로드캐스트 통신이 필요하므로 UDP를 사용한다.

네트워크에 연결된 기기(PC, 스마트폰 등)에 IP 주소, 서브넷 마스크, 게이트웨이, DNS 서버 정보를 자동으로 할당해 주는 네트워크 프로토콜이다.

[QUIC와 HTTP/3]

HTTP/3는 UDP 위에서 구현된 QUIC 프로토콜을 사용한다.

하지만 QUIC가 UDP처럼 신뢰성이 없는 것은 아니다. UDP 위에서 다음 기능을 별도로 구현한다.

- 연결 설정
- 암호화
- 데이터 재전송
- 흐름 제어
- 혼잡 제어
- 스트림 관리

즉, UDP는 QUIC이 운영체제의 TCP 연결 관리에 묶이지 않고 필요한 전송 기능을 사용자 공간에서 구현할 수 있도록 기반을 제공한다.

[12] [UDP 소켓 통신 과정]

[서버]

UDP 서버는 일반적으로 다음 과정을 거친다.

```
socket();
bind();
recvfrom();
sendto();
close();
```

```
socket()
  ↓
bind()
  ↓
recvfrom()로 데이터그램 대기
  ↓
sendto()로 응답
```

TCP 서버와 달리 다음 과정이 없다.

```
listen() 없음
accept() 없음
```

UDP에는 연결 요청을 받아 새로운 연결 소켓을 생성하는 과정이 없기 때문이다.

[클라이언트]

UDP 클라이언트는 일반적으로 다음처럼 동작한다.

```
socket();
sendto();
recvfrom();
close();
```

UDP에서도 connect()를 사용할 수 있지만 TCP의 connect()와 의미가 다르다.

UDP의 connect()는 3-Way Handshake를 수행하지 않는다.

커널에 기본 목적지 IP와 포트를 등록하여 매번 목적지를 지정하지 않고 send()/recv()를 사용할 수 있게 만드는 역할에 가깝다.

TCP connect():

3-Way Handshake를 통해 실제 TCP 연결 설정

UDP connect():

Handshake 없이 기본 통신 상대를 소켓에 지정

[13] [UDP도 포트를 사용하는 이유]

UDP는 연결 상태는 관리하지 않지만 어느 프로세스로 데이터를 전달할지는 구분해야 한다.

예를 들어,

목적지 IP = 192.168.0.10

목적지 Port = 53

IP 주소는 목적지 컴퓨터를 찾고, UDP 포트 번호는 해당 컴퓨터에서 데이터를 받을 프로세스를 찾는다.

IP 주소 → 목적지 호스트 식별

포트 번호 → 목적지 프로세스 식별

TCP와 UDP는 포트 공간이 별도로 관리된다.

따라서 같은 호스트에서 TCP 8080번과 UDP 8080번을 각각 사용할 수 있다.

TCP 8080 ≠ UDP 8080

[14] [UDP에 대한 대표적인 오해]

[UDP는 무조건 TCP보다 빠르다]

UDP는 연결 설정, ACK, 재전송, 흐름 제어 등의 과정이 없어 처리 구조가 단순하다.

따라서 낮은 지연이 필요한 상황에 유리할 수 있다.

하지만 UDP를 사용하는 애플리케이션에서 신뢰성, 재전송, 혼잡 제어를 모두 직접 구현하면 처리 비용이 증가한다. 따라서 모든 환경에서 UDP가 무조건 빠르다고 말할 수는 없다.

[UDP는 데이터가 반드시 손실되는가]

아니다. UDP 데이터도 정상적으로 도착할 수 있다.

UDP는 데이터가 도착하지 않는 것이 아니라, 데이터의 도착을 보장하지 않는다.

[UDP는 오류 검사를 하지 않는가]

UDP는 체크섬을 통해 오류를 검출할 수 있다.

다만 손상되거나 손실된 데이터를 UDP 자체에서 복구하거나 재전송하지 않는다.

[UDP에는 연결이라는 개념이 전혀 없는가]

TCP와 같은 연결 상태 및 Handshake는 없다.

다만, UDP 소켓에도 connect()를 호출해 기본 상대를 지정할 수 있다.